

torch remainder#

<https://pytorch.org/docs/stable/generated/torch.remainder.html#torch.remainder>

Description:

Computes Python's modulus operation entrywise. The result has the same sign as the divisor other and its absolute value is less than that of other. It may also be defined in terms of `torch.div()` as `torch.div(input, other, rounding_mode='floor')`. Supports broadcasting to a common shape, type promotion, and integer and float inputs. Complex inputs are not supported. In some cases, it is not mathematically possible to satisfy the definition of a modulo operation with complex numbers. See `torch.fmod()` for how division by zero is handled.

Function Signature

`torch.remainder(input, other, *, out=None) → Tensor#`

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
torch.remainder(a, b) == a - a.div(b, rounding_mode="floor") * b
```

```
>>> torch.remainder(torch.tensor([-3., -2, -1, 1, 2, 3]), 2)
tensor([ 1.,  0.,  1.,  1.,  0.,  1.])
>>> torch.remainder(torch.tensor([1, 2, 3, 4, 5]), -1.5)
tensor([-0.5000, -1.0000,  0.0000, -0.5000, -1.0000 ])
```

Notes & Warnings

NOTE

Note Complex inputs are not supported. In some cases, it is not mathematically possible to satisfy the definition of a modulo operation with complex numbers. See `torch.fmod()` for how division by zero is handled.

NOTE

See also `torch.fmod()` which implements C++'s `std::fmod`. This one is defined in terms of division rounding towards zero.

Detailed Documentation

Documentation

Computes Python's modulus operation entrywise. The result has the same sign as the divisor `other` and its absolute value is less than that of `other`.

It may also be defined in terms of `torch.div()` as

Supports broadcasting to a common shape, type promotion, and integer and float inputs.

Complex inputs are not supported. In some cases, it is not mathematically possible to satisfy the definition of a modulo operation with complex numbers. See `torch.fmod()` for how division by zero is handled.

`torch.fmod()` which implements C++'s `std::fmod`. This one is defined in terms of division rounding towards zero.

`input` (Tensor or Scalar) – the dividend

`other` (Tensor or Scalar) – the divisor

`out` (Tensor, optional) – the output tensor.